

The Adaptive Company

A Practical Operating System for Human-Agent Work

Gabriel Sarmiento

Contents

	The Adaptive Company	5
1	When Production Gets Cheap	7
1.1	A real change, stated narrowly	7
1.2	Productivity belongs to a system	8
1.3	The constraint moves	9
1.4	Cheap creation has an option cost	11
1.5	Won't agents absorb the new queues?	11
1.6	What to do on Monday	12
2	Hidden Operational Debt	13
2.1	Friction is the payment, not the debt	13
2.2	Not every inconvenience is debt	13
2.3	Six forms of operational debt	14
2.4	Why the debt stays hidden	15
2.5	An illustrative composite: the business line that never leaves the founder	15
2.6	AI can repay debt or capitalize it	15
2.7	Build a debt register, not a complaint list	16
2.8	What to do on Monday	17
	Bibliography	19

The Adaptive Company

A Practical Operating System for Human-Agent Work

Gabriel Sarmiento

AI changes more than the tools people use. It changes how intent becomes action, how authority is delegated, how evidence is produced, and how the company learns.

1. When Production Gets Cheap

A common early failure caused by cheap production is not bad output. It is a company that produces more choices than it can absorb.

Consider a founder-COO in a company of forty people. The business is preparing to add a second activity, hire another thirty people, and build its first real layer of management. A Head of Operations is taking over the daily run. This should release the COO to work on structure, capital allocation, and growth.

It does not.

The new business lead can prepare a market analysis in hours. Finance can generate several pricing and margin scenarios. Product and engineering can prototype different offers in parallel. Each team arrives better prepared. Each also arrives with a decision that crosses the boundaries of the company: which assets remain shared, which capability belongs inside the new unit, what risk the center will carry, and what evidence justifies further investment.

The Head of Operations can run the process. Only the COO still holds enough of the company in their head to resolve the tradeoffs. Work has been delegated. Decisions have not.

This is an **illustrative composite**, not a report about a particular company. Its mechanism is common enough to recognize. At forty people, the founder's intervention can be the shortest path to a good answer. During the move to seventy, then perhaps one hundred and fifty, the same habit becomes part of the architecture. Teams learn to escalate because the founder is fast, informed, and exacting. Managers learn that initiative is welcome until it conflicts with an unstated preference. Shared functions accumulate exceptions. New ventures remain attached to the person who launched them because nobody designed the conditions for transfer to the run.

AI can intensify this dependency. It lowers the effort required to produce an analysis, option, plan, prototype, or recommendation. It does not automatically distribute the authority to choose among them. The organization has not shortened the path from intent to result. It has moved the queue toward judgment, integration, and commitment.

1.1 A real change, stated narrowly

Speed has no organizational meaning until we specify where it occurs and what constrains the complete result. A model can answer in seconds while the company still moves through the same decision, integration, and acceptance cycles. The useful question is not "How fast is AI?" It is "Which constraint moves when this work becomes cheaper?"

The defensible claim is narrow. The marginal cost and latency of producing *some* cognitive and software artifacts have fallen sharply. The emphasis belongs on *some*. The effect varies by task, quality threshold, workflow, and consequence.

One useful indicator is the price of model inference at a fixed level of benchmark performance. Stanford's 2025 AI Index reported that the price for a model performing around the GPT-3.5 level on MMLU fell from about \$20 per million tokens in November 2022 to \$0.07 in October 2024—a decline of more than 280 times [1]. Epoch AI, whose data informed that analysis, found very different rates across tasks and performance thresholds: roughly ninefold to nine-hundredfold annual declines in the cases it examined [2]. The spread matters. There is no single "price of intelligence," and the fastest historical decline should not be projected indefinitely.

Inference price is also not workflow cost. A reliable system may require data access, retrieval, tools, orchestration, evaluation, security, human review, and recovery from failure. A cheap model call can sit inside an expensive operating process.

Economist Joshua Gans offers a useful discipline for thinking about that difference. He treats AI prediction as an input to a decision and examines its value alongside substitutes, complements, automation, and system effects [3]. The point for an operator is straightforward: making one input cheaper does not make the complete decision cheap. The firm still needs data, judgment, and the capacity to act.

Modern generative systems stretch the prediction framing because they can produce artifacts and, when connected to tools, execute actions. The economic discipline still holds. As execution becomes cheaper, its complements become more important. A company needs to specify who may commit resources, what

constraints cannot be bypassed, what evidence must return, and how a bad action can be stopped or reversed. These are not features of the model. They are features of the operating system around it.

Capability has expanded as well. METR’s task-horizon research shows agents completing increasingly long, well-specified tasks in software engineering, machine learning, and cybersecurity [4]. But the researchers warn against turning this into a claim about whole jobs. Their benchmark is concentrated in technical tasks with clear scoring and relatively little organizational context. A 50 per cent success rate is not dependable operation. Messy work, tacit knowledge, competing goals, and institutional consequences change the problem.

The difference between adoption and autonomy is equally important. Stanford’s 2026 AI Index reports AI use in 88 per cent of surveyed organizations, while deployment of agents remains in the single digits across nearly all business functions [5]. The survey does not show that 88 per cent of companies have redesigned work or captured value. It shows that contact with AI is broad while delegated execution remains early.

These findings justify attention. They do not justify declaring the autonomous company inevitable.

1.2 Productivity belongs to a system

“Does AI improve productivity?” is usually too broad a question. Productivity is not an inherent property of a model. It is a result produced by a worker, a task, a tool, a process, and a measure.

In a staggered rollout involving 5,172 customer-support agents, Erik Brynjolfsson, Danielle Li, and Lindsey Raymond found that access to a generative assistant increased issues resolved per hour by 15 per cent on average. The gain was much larger for novice and lower-skilled workers and small for the most experienced workers [6]. That is strong evidence for a specific assisted workflow. It is not evidence that every function becomes 15 per cent more productive, or that the assistant can own the customer relationship.

A randomized study of experienced open-source developers reached a different result. Sixteen developers completed 246 real tasks in repositories they knew well. With early-2025 AI tools, they took 19 per cent longer, even though they believed the tools had made them faster [7]. The sample was small and the result belongs to those developers, repositories, tasks, and tools. It should not become a general claim that coding agents slow engineers.

Nor should it be dismissed because tools improved later. When METR tried to update the estimate in 2026, participation choices, task selection, non-compliance, and concurrent agent work made the treatment effect difficult to identify. The researchers thought newer systems probably offered more acceleration, but described the evidence for the magnitude as weak [8]. The methodological difficulty is itself instructive: once people delegate, supervise several runs, or choose different work because an agent is available, keyboard time stops being an adequate measure.

The support and software studies do not cancel each other. They describe different systems. One offered recommendations inside a high-volume workflow with observable outcomes. The other involved experienced maintainers changing mature codebases where understanding, integration, and review are substantial parts of the work. A person may become faster at a task while the system around that person remains unchanged—or becomes slower under the added load.

DORA’s 2025 research on software delivery describes AI as an amplifier of the surrounding organization: strong systems can convert local acceleration into performance, while weak ones can produce instability faster [9]. That finding is observational, not proof of a universal law. It nevertheless points toward the right unit of analysis. Faster generation alone does not tell us how much organizational value was created.

We need four different words:

Term	Meaning	A misleading proxy
Output	An artifact or action produced: code, analysis, copy, a recommendation, a transaction	Number of artifacts generated
Throughput	Accepted work that crosses the complete operating path	Tasks started or drafts produced
Outcome	A change in customer, operational, or strategic conditions	Work shipped without measured effect

Term	Meaning	A misleading proxy
Value	The benefit of that outcome after cost, risk, maintenance, and foregone alternatives	Model cost or hours reportedly saved

Cheap output is useful. It is not the same as throughput, outcome, or value. The difference matters to a COO because the company pays for the complete system while each team tends to report the part it produced.

1.3 The constraint moves

At any moment, end-to-end performance is limited by one or more constraints. Expanding capacity at one constraint is valuable until another limits the result. If a company can generate analyses, designs, code, and communications much faster without changing how it chooses, integrates, checks, and authorizes them, scarcity moves elsewhere.

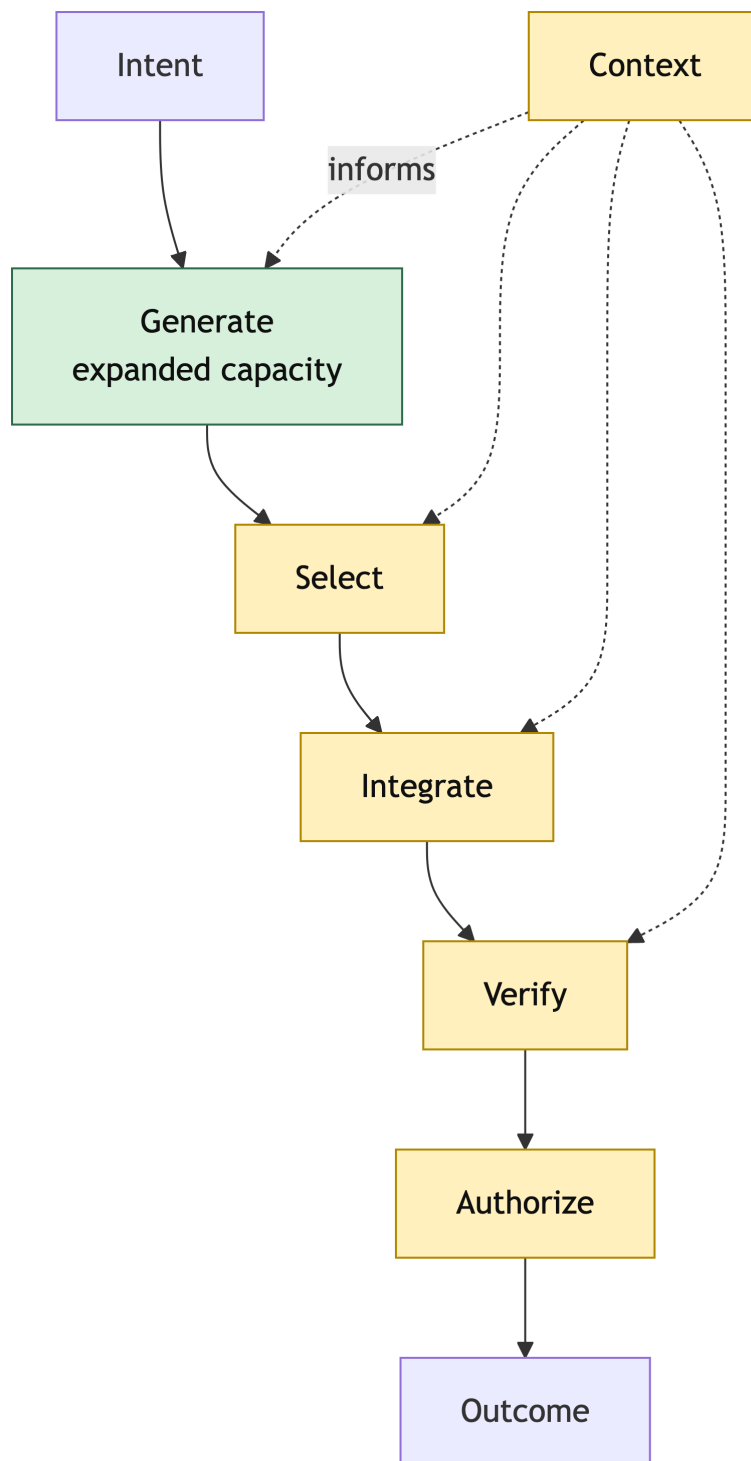


Figure 1.1: When generation capacity expands, the yellow capacities can become queues in the surrounding operating system.

The exact sequence varies. Context can shape generation or arrive later to test an option. Verification may run continuously. Low-risk actions may be authorized in advance. The diagram is not a workflow template. It identifies five capacities that cheap production can expose.

None of these capacities is new. What changes is the imbalance between them. Generative systems can increase the arrival rate of work without increasing the capacity of teams that must absorb it. Agents can

also participate in the downstream steps, changing who or what selects, checks, and acts. The constraint can therefore move faster and become harder to see in conventional activity measures.

Selection. Which request deserves work? Which option deserves commitment? Generation creates alternatives; it does not make the opportunity cost of choosing one disappear. A portfolio of cheap experiments can still consume the scarce attention needed to stop most of them.

Context. Does the work reflect the organization's current intent, facts, constraints, history, and policy? A fluent answer built from stale commercial terms or the wrong customer state is fast rework. Supplying context is not a one-time prompt-writing exercise. Context must be current, accessible, permitted, and traceable.

Integration. Can the output enter a product, process, data environment, or customer relationship without transferring work to someone else? A generated feature that creates a new service dependency is not finished when its code runs. A proposal that cannot survive finance, security, or delivery constraints is not organizational throughput.

Verification. What evidence is proportionate to the consequence? A private brainstorming note and a payment instruction should not share an evaluation regime. Verification can include tests, source checks, simulations, peer review, monitoring, or comparison with a known baseline. It need not mean inspecting everything manually. It should explain why the system is trusted enough to act.

Authorization. Who—or what—may publish, deploy, spend, promise, reject, or bind the company? An agent may be technically capable of taking an action without having legitimate authority to take it. Authorization converts capability into bounded institutional power.

Accountability surrounds all five. It is not another box through which work passes. It answers a different question: who remains answerable for the outcome when work is divided among people, models, software, and vendors?

For the scaling COO, these queues often collapse into one calendar. Selection arrives as portfolio review. Missing context arrives as a request to recall why an earlier promise was made. Integration arrives as conflict between a business unit and a shared function. Verification arrives as a document that needs “one final look.” Authorization arrives as a message asking, “Are you comfortable if we proceed?”

The founder experiences these as unrelated interruptions. They are often one design failure: the organization depends on a person where it needs a decision system.

1.4 Cheap creation has an option cost

The apparent price of an additional option approaches zero long before its total cost does.

Return to the growing company. The fourth plan for the new business line may cost almost nothing to generate. It still asks the leadership team to compare assumptions, resolve conflicts with the existing business, test the economics, decide which capabilities remain shared, and state who may commit the company. Even a rejected plan can leave residue: forecasts, prototypes, provisional hires, technology choices, stakeholder expectations, and exceptions that somebody must later unwind.

A more honest cost equation is:

Total system cost = generation + selection + context + integration + verification + authorization + failure recovery + maintenance.

This is not an accounting standard. It is a checklist for costs that pilots often omit. If a pilot records only license fees and minutes saved, it can report a local gain while exporting cost to reviewers, platform teams, risk owners, or future maintainers.

The opposite mistake is to demand that every use case prove enterprise-wide value before it begins. Learning also has value. A small experiment can be justified by the uncertainty it resolves. But that requires an explicit learning question and a stop decision. “We can generate it” is not a reason to keep it.

1.5 Won't agents absorb the new queues?

They will absorb parts of them. Agents can rank alternatives, retrieve context, open pull requests, run evaluations, and act within pre-approved limits. As capability improves, the constraint will move again.

That does not make operating design temporary. It makes operating design continuous.

An agent selecting investments still needs an objective and evidence of decision quality. An agent integrating code still needs access boundaries, tests, and a recoverable deployment path. An agent approving a refund has been given authority that must have a source, a limit, and a record. More capable execution

changes where humans intervene; it does not remove the need to decide how intent, authority, evidence, and learning move through the company.

The claim also has boundaries. It is strongest in digital, decomposable, repeated work with observable results. Physical constraints may dominate manufacturing or logistics. Trust and relationship may dominate negotiation or care. Ambiguous strategy cannot be made sound merely by generating more strategic options. Stable processes should not be made fluid for the sake of appearing modern.

The point is not that production no longer matters. It is that production can no longer be assumed to be the limiting step.

1.6 What to do on Monday

Choose one recurring decision that still returns to the founder or COO. Use a real decision: a price exception, senior hire, product commitment, capital request, customer promise, or dispute between a business unit and a shared function.

1. Name the decision and the business outcome it is meant to protect. Do not name a meeting or document.
2. Write down the formal owner, then the person who makes the decision in practice.
3. Trace where the decision waits: selection, context, integration, verification, or authorization.
4. Record what the COO knows, checks, or weighs that the stated owner cannot yet access or apply.
5. Design one delegation test for the next occurrence: a clear intent, decision boundary, evidence requirement, and escalation condition. Do not silently retake the decision when the answer differs from your preference.

This exercise will often reveal a paradox: the organization is producing more while becoming harder to steer. Decisions multiply. Dependencies accumulate. Review work arrives without capacity. Exceptions live in messages and meetings. The COO remains indispensable not only because others lack skill, but because the company has stored context, judgment, and authority in one person.

That is not solved by telling the COO to let go. It is solved by making the missing operating structure explicit.

That liability is **Hidden Operational Debt**. It is where the next chapter begins.

2. Hidden Operational Debt

The slow decision from Chapter 1 is rarely slow because nobody worked on it.

The business-unit lead prepared the recommendation. Finance checked the margin. Operations assessed capacity. An AI assistant summarized the alternatives and surfaced the main risks. Yet the decision still returned to the COO.

The organization calls this an approval delay. That describes where the work stopped. It does not explain why.

The COO may hold context that never entered the analysis: a promise made to an investor, a constraint in a supplier relationship, the reason an earlier pricing exception failed, or an unresolved disagreement between two executives. The formal owner can prepare the decision but cannot safely make it. The meeting is not the underlying problem. It is an interest payment on an operating liability.

That liability is **Hidden Operational Debt**.

2.1 Friction is the payment, not the debt

The debt metaphor comes from software engineering. Technical debt describes a design compromise that may accelerate work now while making later change more difficult or expensive. Research has since extended the metaphor to structures, policies, processes, and social arrangements, although the evidence on “organizational debt” remains young and concentrated in software settings. A 2024 multivocal review found only nine academic papers and supplemented them with practitioner writing. Its recurring themes included outdated structures, deferred decisions, siloed work, and the continuing cost of arrangements that no longer serve the organization [10]. A small case study of large-scale hybrid software development similarly connected organizational debt with weak shared mental models, coordination, cohesion, and team learning [11]. These studies make the neighboring concept credible; they do not establish a universal law of organizations.

Hidden Operational Debt is narrower and more usable:

Hidden Operational Debt is an unresolved operating-model liability that repeatedly imposes delay, rework, coordination cost, or risk when the company tries to decide, execute, or change.

Four parts matter.

The liability is the unresolved design problem. Decision rights are ambiguous. Critical context lives in one person’s memory. Two systems require manual reconciliation. A shared function and a business unit have no stable interface.

The interest is the recurring cost of leaving that liability in place. People wait, reconstruct context, attend another meeting, correct duplicated work, request an exception, or escalate to someone with enough authority to proceed.

The risk exposure is what can happen while the liability persists. A margin exception becomes the commercial norm. A regulator cannot reconstruct an approval. A customer receives two incompatible commitments. The company launches a new activity without knowing which costs it truly owns.

The principal is the work required to change the underlying arrangement. That may mean clarifying one decision right, redesigning an interface, consolidating a source of truth, retiring a workaround, or changing a leader’s behavior. Principal is not the total cost of perfection. It is the smallest coherent change that stops the recurring charge.

This is a management model, not an accounting standard. The point is to distinguish the visible irritation from the condition that keeps producing it.

2.2 Not every inconvenience is debt

Calling every slow or disliked process “debt” makes the term useless.

A deliberate control can create friction and still be valuable. Requiring two people to authorize a large payment slows execution because the consequence warrants separation of duties. A stable manufacturing process may resist rapid change because variation would introduce safety or quality risk. A founder may keep a rare acquisition decision because the board expects that accountability.

Debt exists when the recurring cost comes from an unresolved mismatch rather than a justified design choice. Three tests help:

1. **Recurrence:** Does the same class of work keep paying the cost?
2. **Avoidability:** Could a change to context, rights, interfaces, controls, or learning remove the cost without creating greater harm?
3. **Persistence:** Has the organization allowed a temporary compromise to become the normal operating path?

One difficult quarter does not prove debt. A repeated pricing escalation caused by an unstated margin rule probably does.

Debt can also be intentional. Early in a company, asking the founder to approve every non-standard contract may be rational. The company has little precedent, few managers, and no reliable way to encode commercial judgment. The mistake is not borrowing. The mistake is forgetting that the arrangement was temporary.

A conscious operating shortcut should therefore have an owner, a reason, a review date, and a condition for repayment. Without those, an expedient decision quietly becomes architecture.

2.3 Six forms of operational debt

The categories below are diagnostic. One liability can create several kinds of interest, so the aim is not perfect classification. The aim is to locate what must change.

Debt type	Underlying liability	Typical interest payment	Scaling symptom
Decision debt	Rights, criteria, thresholds, or escalation paths are unclear	Repeated approvals, reversals, executive arbitration	Managers execute work but defer consequential choices
Context debt	Current facts, constraints, policies, and state are inaccessible or untrusted	Reconstructing the situation before every decision	The person with the longest memory becomes mandatory
Coordination debt	Dependencies lack a stable mechanism or cadence	Status meetings, chasing, manual synchronization	Adding teams creates more alignment work than capacity
Integration debt	Technical or organizational interfaces are brittle, bespoke, or undefined	Rework, reconciliation, hand-built handoffs	Each new product or business unit creates exceptions in shared functions
Knowledge debt	Rationale, outcomes, and lessons are not retained in reusable form	Repeating analysis, re-discovering failure, inconsistent onboarding	Growth dilutes judgment instead of distributing it
Control debt	Permissions, guardrails, evidence, or exception handling are improvised	Manual checking, broad access, approval theater, weak audit trails	The company chooses between paralysis and unsafe autonomy

The distinctions reveal different remedies. A weekly cross-functional meeting may be coordination debt if no one can see dependencies. It may be decision debt if the meeting exists because nobody can commit. It may be context debt if the first forty minutes are spent establishing which numbers are true. Canceling the meeting treats all three as calendar clutter and resolves none of them.

2.4 Why the debt stays hidden

Financial debt appears on a balance sheet. Operational debt is dispersed.

One team waits. Another team reconciles. Finance rebuilds a model. A manager prepares a pre-meeting so that the real meeting will succeed. The COO answers messages after dinner. Each cost looks small from where it is incurred, and no single budget line shows the total.

Competent people hide the debt further. They remember the exception, call the right colleague, correct the spreadsheet, and obtain the approval. The company interprets the rescued outcome as proof that the process works. In fact, it works because someone repeatedly pays the interest with attention and reputation.

Growth can turn this competence into a trap. The founder who resolves ambiguity fastest receives more ambiguity. The shared-function leader who knows how to navigate every exception becomes the route for every exception. The engineer who understands the integration is added to every project. Their usefulness increases while the organization's ability to operate without them declines.

This is why measuring only activity is dangerous. More completed analyses can coexist with longer decision latency. More launches can coexist with weaker unit economics. More delegation can coexist with the founder retaining every meaningful right. A dashboard shows the outputs and misses the dependence that produced them.

2.5 An illustrative composite: the business line that never leaves the founder

Consider the company from Chapter 1. The founder-COO personally launches a second business line because uncertainty is high and speed matters. The choice is reasonable. The founder negotiates the first partnerships, approves pricing, decides which capabilities to borrow from the existing business, and uses personal judgment to protect cash.

The activity begins to work. A business-unit lead is appointed. The Head of Operations takes over delivery. The organization chart says the venture has moved into the run.

It has not.

Pricing exceptions still return to the founder because the unit has no explicit margin floor and finance cannot see the full cost of shared assets. Hiring returns because nobody has defined which roles belong inside the unit. Product decisions return because the new activity shares a platform with the original business. Commercial commitments return because only the founder knows which early promises were deliberate and which were experiments.

The visible symptoms are an overloaded COO, slow approvals, and a business-unit lead who appears hesitant. The easy diagnosis is that the leader needs to become more decisive. The harder diagnosis is that the company has assigned accountability without transferring the context and authority required to carry it.

The principal is not "reorganize the company." It is a set of specific design choices:

- define which economic result the unit owns;
- expose the cost and capacity of shared assets;
- state which decisions belong to the unit, shared functions, and the center;
- convert early promises and exceptions into current policy or retire them;
- specify when a decision must escalate and what evidence accompanies it;
- let the delegated owner make a sound decision the founder would not personally prefer.

That last item matters. A founder can repay the formal debt and continue charging interest through behavior. If every different answer triggers a review, the real decision right never moved.

2.6 AI can repay debt or capitalize it

AI does not have a fixed relationship with operational debt. It can reduce an interest payment while leaving the liability untouched. It can help remove the liability. It can also make the debt larger and less visible.

Debt type	AI can help repay it by...	AI can increase it by...
Decision	Structuring options, applying explicit thresholds, recording rationale	Producing more options, concealing weak criteria behind fluent recommendations
Context	Retrieving authorized facts with sources and freshness	Repeating stale summaries, mixing policies, stripping provenance
Coordination	Summarizing state, routing exceptions, exposing dependencies	Starting more parallel work and generating more requests for other teams
Integration	Building adapters, tests, and repeatable handoffs	Proliferating brittle code, tools, data copies, and local workarounds
Knowledge	Capturing decisions and linking outcomes to prior reasoning	Filling repositories with plausible but unverified documents
Control	Enforcing policy, checking permissions, and preserving evidence	Expanding credential access, action volume, and review burden faster than controls

The distinction between paying interest and repaying principal is essential. An assistant that writes the weekly status report may save a manager two hours. If the report exists because five teams lack a shared operating view, the assistant has made the interest cheaper. The liability remains.

That can still be worthwhile. There is no virtue in paying interest manually. But leaders should know which result they bought. Automating the workaround is not the same as redesigning the system.

The more dangerous case is new borrowing: debt created in the name of acceleration. A team uses agents to launch several experiments, each with a new data flow and no owner for retirement. A business unit builds its own customer definitions because retrieving the shared ones is slow. An approval agent receives broad permissions because the policy model is incomplete. Local output rises. Future change becomes more expensive.

AI makes disciplined debt management more important because it increases the rate at which temporary arrangements can be created.

2.7 Build a debt register, not a complaint list

A useful register connects a symptom to a liability and a testable repayment. It does not collect everything employees dislike.

Use these fields:

Field	Question
Trigger	What recurring event causes the cost to appear?
Symptom	What do people observe: waiting, rework, escalation, exception, or failure?
Liability	Which missing or outdated design choice creates the symptom?
Interest	What time, delay, cost, or attention is paid on each occurrence?
Risk	What consequence becomes more likely while the liability remains?
Owner	Who can change the underlying operating arrangement?

Field	Question
Repayment experiment	What is the smallest coherent change that could reduce recurrence?
Evidence	What result would show repayment rather than displacement of the cost?
Review date	When will the company keep, revise, or reverse the change?

For the pricing example, the entry might read:

- **Trigger:** a business-unit deal outside the standard price list;
- **Symptom:** the unit waits for the COO after finance has already reviewed the deal;
- **Liability:** no delegated discount envelope tied to unit margin, capacity, and contract risk;
- **Interest:** repeated analysis, executive interruption, commercial delay, and learned hesitation;
- **Risk:** inconsistent discounts or growth that destroys unit economics;
- **Owner:** COO with the business-unit and finance leads;
- **Repayment experiment:** delegate decisions inside a stated margin and risk envelope for thirty days;
- **Evidence:** decision time, escalation rate, achieved unit margin, reversals, and exceptions;
- **Review date:** after the first review cycle, not after an arbitrary transformation milestone.

The experiment can fail. Perhaps shared-asset cost is too unreliable to support the threshold. That result is useful: the apparent decision debt depends on context or integration debt. The register should then change. Debt categories are a route to investigation, not labels to defend.

2.8 What to do on Monday

Take the recurring decision identified at the end of Chapter 1. Review its last few occurrences.

1. Write one sentence: “Because [**liability**], every [**trigger**] requires [**interest payment**], exposing us to [**risk**].”
2. Separate necessary control from avoidable friction. State what harm the current step prevents.
3. Identify the smallest change to a right, context source, interface, control, or learning record that could stop recurrence.
4. Choose evidence from the complete path: decision time, touches, rework, reversals, exceptions, outcome, and relevant unit economics.
5. Give the experiment an owner and review date. If it is temporary, say when the company will repay or renew it.

Do not begin with a company-wide debt score. A precise register of five consequential liabilities is more useful than a survey producing one alarming number. Do not begin with a reorganization. Changing boxes before identifying the liability often moves the interest to a different team.

Most importantly, do not ask only, “What can AI automate?” Ask, “Which liability would still exist after the automation?”

That question separates a useful tool from an operating-model change. It is the subject of the next chapter.

Bibliography

- [1] Stanford Institute for Human-Centered Artificial Intelligence, “Artificial Intelligence Index Report 2025,” 2025. Accessed: Sep. 11, 2026. [Online]. Available: https://hai.stanford.edu/assets/files/hai_ai-index-report-2025_chapter1_final.pdf
- [2] B. Cottier, B. Snodin, D. Owen, and T. Adamczewski, “LLM Inference Price Trends.” Accessed: Sep. 11, 2026. [Online]. Available: <https://epoch.ai/data-insights/llm-inference-price-trends>
- [3] J. Gans, *The Microeconomics of Artificial Intelligence*. The MIT Press, 2025. doi: [10.7551/mitpress/15248.001.0001](https://doi.org/10.7551/mitpress/15248.001.0001).
- [4] Model Evaluation & Threat Research, “Task-Completion Time Horizons of Frontier AI Models,” 2026. Accessed: Sep. 11, 2026. [Online]. Available: <https://metr.org/time-horizons/>
- [5] Stanford Institute for Human-Centered Artificial Intelligence, “The 2026 AI Index Report,” Apr. 2026. Accessed: Sep. 11, 2026. [Online]. Available: <https://hai.stanford.edu/ai-index/2026-ai-index-report>
- [6] E. Brynjolfsson, D. Li, and L. R. Raymond, “Generative AI at Work,” *The Quarterly Journal of Economics*, vol. 140, no. 2, 2025, doi: [10.1093/qje/qjae044](https://doi.org/10.1093/qje/qjae044).
- [7] J. Becker, N. Rush, B. Barnes, and D. Rein, “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” Jul. 2025. Accessed: Sep. 11, 2026. [Online]. Available: https://metr.org/Early_2025_AI_Experienced_OS_Devs_Study-paper.pdf
- [8] Model Evaluation & Threat Research, “Update: How Much Does AI Speed Up Experienced Developers?” Accessed: Sep. 11, 2026. [Online]. Available: <https://metr.org/blog/2026-02-24-uplift-update/>
- [9] DORA, “State of AI-Assisted Software Development 2025,” 2025. Accessed: Sep. 11, 2026. [Online]. Available: <https://dora.dev/research/2025/dora-report/>
- [10] O. Al-Baik, M. Abu Alhija, H. Abdeljaber, and M. Ovais Ahmad, “Organizational Debt—Roadblock to Agility in Software Engineering: Exploring an Emerging Concept and Future Research for Software Excellence,” *PLOS ONE*, vol. 19, no. 11, p. e0308183, Nov. 2024, doi: [10.1371/journal.pone.0308183](https://doi.org/10.1371/journal.pone.0308183).
- [11] Z. Liu, V. G. Stray, and T. T. Sporse, “Organizational Debt in Large-Scale Hybrid Agile Software Development: A Case Study on Coordination Mechanisms,” in *Agile Processes in Software Engineering and Extreme Programming—Workshops*, Springer, 2023, pp. 75–84. doi: [10.1007/978-3-031-48550-3_8](https://doi.org/10.1007/978-3-031-48550-3_8).